

Relatório de Desenvolvimento de Software:
NL to SQL – Conversor de Linguagem Natural
para SQL utilizando LLM Local e PySide6

Aluno: João Victor Mendes de Siqueira

Sumário

1	Introdução	2
2	Tecnologias Utilizadas	2
3	Arquitetura do Sistema e Fluxo de Execução	2
4	Metodologia de Conversão de Linguagem Natural para SQL	3
5	Análise Técnica dos Componentes de Código	4
5.1	Ponto de Entrada Principal (main.py)	4
5.2	Camada de Interface Gráfica (gui/)	4
5.3	Componentes de Integração com Modelos de Linguagem (llm/)	4
5.4	Mecanismo de Persistência e Banco de Dados (DataBase/)	5
6	Instalação e Configuração do Ambiente	5
6.1	Preparação do Ambiente Python e Dependências	5
6.2	Configuração do Servidor do Modelo Local (Ollama)	6
7	Requisitos de Hardware	6
8	Manual de Utilização do Software	7
8.1	Avisos e Limitações de Uso	8
9	Validação Experimental	8
10	Conclusão	11

1 Introdução

Este relatório descreve o desenvolvimento do sistema **NL to SQL Converter**, uma aplicação desktop projetada para receber perguntas em linguagem natural, traduzi-las automaticamente para comandos SQL válidos através de um modelo de inteligência artificial executado localmente, e realizar a execução direta em servidores de banco de dados relacionais.

2 Tecnologias Utilizadas

O desenvolvimento da aplicação exigiu a integração de tecnologias voltadas para interface gráfica, acesso a bancos de dados relacionais e inteligência artificial generativa. A Tabela 1 apresenta os principais recursos empregados.

Tabela 1: Tecnologias utilizadas no projeto

Tecnologia	Finalidade
Python 3	Linguagem principal do projeto
PySide6	Interface gráfica desktop
SQLAlchemy	Abstração e conexão com bancos relacionais
PyMySQL	Driver MySQL
psycopg2	Driver PostgreSQL
Ollama	Execução local de modelos LLM
Qwen3:14B	Conversão de linguagem natural para SQL
Git e GitHub	Controle de versão e hospedagem do código

A utilização do Ollama permitiu que toda a inferência fosse executada localmente, eliminando a necessidade de serviços externos.

3 Arquitetura do Sistema e Fluxo de Execução

O sistema foi concebido seguindo princípios de design modular, desacoplamento de responsabilidades e garantia de responsividade de interface gráfica. Operações que envolvem inferência de redes neurais artificiais e comunicação em rede com SGBDs possuem tempo de resposta variável e tendem a bloquear a execução sequencial do programa.

Para solucionar esse problema, o sistema adota uma arquitetura orientada a eventos assíncronos apoiada no modelo de *Multithreading* do PySide6. A interface gráfica opera estritamente na thread principal (*Main Thread*), enquanto todo o pipeline de processamento de inteligência artificial e execução de consultas é delegado a uma thread de segundo plano gerenciada por um operário (*SQLWorker*).

O fluxo de processamento completo da aplicação compreende as seguintes etapas sequenciais:

1. **Configuração e Conexão do Banco de Dados:** O ciclo operacional inicia-se obrigatoriamente na janela de configuração (*DatabaseConfigDialog*). O usuário fornece as credenciais do servidor (Tipo do SGBD, Host, Porta, Usuário, Senha e

Database) e realiza o teste de conexão. O sistema valida as informações em tempo real via `SQLAlchemy` e armazena os parâmetros validados na memória da aplicação.

2. **Entrada em Linguagem Natural:** Com a comunicação com o banco de dados devidamente estabelecida e configurada, a interface principal (`MainWindow`) é liberada. O usuário insere sua pergunta em português corrente no campo de texto e aciona o comando de consulta.
3. **Delegação Assíncrona:** A pergunta digitada e o dicionário contendo as configurações de conexão salvas na primeira etapa são empacotados juntos e despachados para o `SQLWorker`. Este componente cria e inicia uma nova linha de execução isolada (`QThread`), evitando que a tela principal trave durante o processamento.
4. **Extração Dinâmica de Esquema:** Já operando em segundo plano (na nova thread), o operário invoca o componente `SchemaLoader`, que utiliza os parâmetros de conexão recebidos para ler o catálogo do banco em tempo real, extraíndo os nomes exatos de todas as tabelas e colunas disponíveis.
5. **Construção do Prompt e Inferência (LLM):** O esquema extraído é convertido em um formato textual padronizado e injetado pelo `OllamaClient` em um prompt de sistema estrito. O servidor local do `Ollama` processa o prompt utilizando o modelo `qwen3:14b` e retorna uma string contendo exclusivamente o comando SQL gerado, sem blocos de texto explicativos.
6. **Execução da Query e Renderização:** O comando SQL resultante é repassado ao `QueryExecutor`, que abre uma sessão segura e executa a instrução nativamente no banco de dados. Ao obter as linhas e colunas de resposta, o `SQLWorker` emite um sinal de término (`finished`) repassando os dados estruturados de volta para a thread principal, que atualiza o componente tabular (`QTableWidget`) e exibe o código gerado para fins de auditoria.

4 Metodologia de Conversão de Linguagem Natural para SQL

A geração automática de consultas SQL foi implementada utilizando a abordagem conhecida como *Text-to-SQL*. Nessa técnica, um modelo de linguagem recebe simultaneamente a pergunta formulada pelo usuário e uma descrição textual do esquema do banco de dados.

Antes de cada consulta, o sistema utiliza os mecanismos de reflexão (*reflection*) disponibilizados pelo `SQLAlchemy` para identificar automaticamente todas as tabelas e colunas presentes na base de dados conectada.

Essas informações são convertidas para um formato textual simplificado, semelhante ao exemplo abaixo:

```
1 DATABASE SCHEMA
2
3 TABLE student
4 - ID
5 - name
6 - dept_name
7
```

```

8 TABLE department
9 - dept_name
10 - building
11 - budget
12
13 Pergunta:
14 Quais alunos pertencem ao departamento de Física?

```

O esquema e a pergunta são enviados conjuntamente ao modelo Qwen3:14B. Dessa forma, o modelo possui conhecimento explícito sobre a estrutura da base de dados e consegue selecionar corretamente tabelas, relacionamentos e filtros necessários para construir a consulta SQL.

Uma das principais vantagens dessa abordagem é a independência em relação ao banco utilizado. Como o esquema é extraído dinamicamente a cada execução, o sistema pode ser aplicado a diferentes bases de dados sem necessidade de retreinamento do modelo.

5 Análise Técnica dos Componentes de Código

O ecossistema do projeto é subdividido em módulos especializados localizados nos diretórios `gui/`, `llm/` e `DataBase/`. A seguir, detalham-se os papéis e comportamentos dos arquivos estruturais que compõem a aplicação.

5.1 Ponto de Entrada Principal (`main.py`)

O arquivo `main.py` atua como o inicializador do ciclo de vida da aplicação. Ele instancia a classe global de gerenciamento do PySide6 (`QApplication`), inicializa a janela gráfica principal do sistema (`MainWindow`), exibe-a em tela e transfere o controle do processamento para o laço interno de eventos da interface.

5.2 Camada de Interface Gráfica (`gui/`)

A interface é composta por duas janelas com responsabilidades de interação bem definidas:

- **MainWindow.py:** Gerencia os elementos principais de tela, incluindo o campo de entrada textual de perguntas, o terminal de *log* e status, a caixa de exibição do SQL estruturado gerado pela IA e um componente `QTableWidget` configurado dinamicamente no modo `QHeaderView.Stretch` para acomodar e redimensionar automaticamente qualquer volume de colunas e linhas resultantes das tabelas do banco.
- **DataBaseConfigWindow.py:** Implementa o diálogo `DatabaseConfigDialog`. Sua função principal é coletar credenciais de acesso (Host, Porta, Usuário, Senha e Nome do Banco) para MySQL ou PostgreSQL. Adicionalmente, integra uma rotina de teste de conexão em tempo real que, através da função `inspect()` do SQLAlchemy, verifica a validade das credenciais e alerta imediatamente o usuário sobre a lista de tabelas existentes ou falhas de comunicação.

5.3 Componentes de Integração com Modelos de Linguagem (`llm/`)

A inteligência e o controle de geração do SQL são governados por dois módulos acoplados:

- **OllamaClient.py:** Interface direta de conversação com o *backend* do Ollama. Este componente contém a **E engenharia de Prompt** crítica do sistema. O prompt estruturado isola as tabelas do banco por meio de variáveis de contexto e impõe regras semânticas rígidas ao modelo de linguagem, garantindo que o retorno seja limpo de formatações de texto comuns em chats (como blocos de marcação de markdown ““sql) e impedindo o modelo de emitir explicações textuais adicionais.
- **SqlWorker.py:** Uma subclasse de `QObject` projetada para operar de forma assíncrona. Ela implementa o método `run()` que atua como o roteador central do pipeline, consumindo os métodos de extração, inferência e execução. O operário gerencia o tratamento de erros em nível de execução e utiliza o sistema de sinais nativos (`finished` e `error`) para garantir uma comunicação segura entre threads.

5.4 Mecanismo de Persistência e Banco de Dados (DataBase/)

O acesso a dados e a acesso aos dialetos SQL é feito através de três rotinas específicas:

- **connection.py:** Implementa a classe `DatabaseConnection` utilizando o padrão *Factory*. A partir do dicionário de dados gerado pela janela de configuração, o método monta de forma transparente a string de conexão específica (*Data Source Name - DSN*), mapeando o driver `pymysql` para bancos de dados MySQL e o driver `psycopg2` para servidores PostgreSQL, retornando o objeto `Engine` do SQLAlchemy.
- **SchemaLoader.py:** Abstrai o mapeamento dinâmico de estruturas do banco. O método `load_schema` varre o catálogo do banco de dados coletando os nomes das tabelas e as respectivas colunas. O método complementar `schema_to_text` formata essas listas em uma estrutura textual padronizada que serve de insumo para o modelo de contexto da IA.
- **QueryExecutor.py:** Encapsula a execução de queries textuais sob uma sessão segura. Utilizando o bloco gerenciador de contexto (`with engine.connect() as conn`), o executor converte a string enviada pela IA em uma expressão interpretável via `text(sql)`, realiza a chamada no servidor de banco de dados e separa metadados de colunas das linhas de dados brutas (`result.fetchall()`).

6 Instalação e Configuração do Ambiente

A implantação do ambiente do sistema exige a configuração das dependências em Python e a ativação do servidor de inteligência artificial local.

6.1 Preparação do Ambiente Python e Dependências

O projeto requer o interpretador Python 3 instalado. Recomenda-se o isolamento de dependências por meio de ambientes virtuais. O arquivo `requirements.txt` define as seguintes bibliotecas obrigatórias para a execução do ecossistema do software:

```
1 ollama
2 sqlalchemy
3 pymysql
4 psycopg2-binary
5 pandas
```

Listing 1: Conteúdo do arquivo requirements.txt

O procedimento completo de clonagem de repositório, criação do ambiente virtual e instalação automatizada das bibliotecas via gerenciador de pacotes `pip` em ambiente Linux é apresentado a seguir:

```

1 # 1. Clonar o repositório git do projeto
2 git clone https://github.com/JoaoVictor-ec/nl_to_sql_llm.git
3
4 # 2. Criar e ativar o ambiente virtual isolado (Linux)
5 python3 -m venv nl_sql
6 source nl_sql/bin/activate
7
8 # 3. Instalar o conjunto de dependências obrigatórias
9 pip install -r requirements.txt

```

Listing 2: Comandos para Instalação e Configuração do Ambiente

6.2 Configuração do Servidor do Modelo Local (Ollama)

Com o ambiente Python estabelecido, torna-se necessária a instalação do utilitário Ollama, responsável pela execução nativa com aceleração de hardware dos modelos de linguagem. O instalador para diferentes sistemas operacionais (Linux, Windows e macOS) encontra-se disponível no portal oficial: <https://ollama.com/>.

Após a instalação bem-sucedida do serviço do Ollama no sistema operacional, deve-se realizar o *download* local dos pesos do modelo `qwen3:14b`, parametrizado nativamente no construtor da classe `OllamaClient`:

```

1 # Baixar o modelo de parametrizacao de 14 bilhoes de variaveis
2 ollama pull qwen3:14b

```

Listing 3: Comando para Inicialização e Download do Modelo

7 Requisitos de Hardware

Embora o sistema possa se beneficiar da utilização de GPU durante a execução do modelo de linguagem, o projeto não implementa mecanismos específicos de aceleração por GPU. Assim provando que mesmo sem o uso da placa de vídeo ainda é possível executá-lo em uma máquina com 16 GB de Ram.

A configuração utilizada durante o desenvolvimento do projeto é apresentada na Tabela 2.

Tabela 2: Ambiente utilizado durante o desenvolvimento

Sistema Operacional	Ubuntu 22.04 LTS
Processador	Intel Core i7
Memória RAM	16 GB
GPU	NVIDIA RTX 4050 (6 GB VRAM)
Python	3.x
Modelo LLM	Qwen3:14B

Durante os testes observou-se que consultas simples são geradas em poucos segundos utilizando a configuração descrita, mantendo a interface responsiva graças ao uso de processamento assíncrono por meio de `QThread`. Enquanto consultas mais complexas levam um tempo maior.

8 Manual de Utilização do Software

A operação do sistema é sequencial e guiada pela interface gráfica. O fluxo operacional para execução de consultas estruturadas compreende os passos descritos abaixo:

1. **Inicialização do Sistema:** Com o ambiente virtual devidamente ativado e o Ollama instalado juntamente com o modelo Qwen3:14B, execute o ponto de entrada principal do programa pelo terminal do sistema:

```
1 python main.py
```

Durante a execução, a aplicação utiliza a API local do Ollama para realizar inferências com o modelo de linguagem, responsável pela geração das consultas SQL a partir das perguntas fornecidas pelo usuário.

2. **Configuração do Banco de Dados:** Na tela inicial da aplicação, localize e clique no botão de ação **Configurar Banco**. Uma janela de diálogo dedicada será sobreposta.
 - Selecione o tipo de banco de dados desejado na caixa de seleção (*MySQL* ou *PostgreSQL*).
 - Preencha os campos correspondentes com as informações de rede do servidor: *Host* (padrão: *localhost*), *Porta* (padrão: *3306* para *MySQL*), além de *Usuário*, *Senha* e o nome exato da *Database* que deseja mapear.
3. **Validação de Credenciais:** Na mesma janela de diálogo de configuração, clique na opção **Testar Conexão**. O sistema utilizará os módulos do SQLAlchemy para tentar efetuar um login e ler os metadados. Caso os parâmetros estejam corretos, uma caixa informativa de sucesso listará todas as tabelas encontradas na base de dados. Clique em **Salvar** para persistir as definições na memória da janela principal.
4. **Consulta em Linguagem Natural:** Com a conexão estabelecida, vá até a caixa de texto de entrada principal da interface e redija uma pergunta direta sobre os dados corporativos em português corrente.
 - *Exemplo de entrada:* “Quais clientes fizeram mais pedidos?” ou “Qual o faturamento total acumulado no último mês?”.
5. **Processamento e Visualização:** Clique no botão de comando **Consultar**. O sistema bloqueará temporariamente o botão para evitar cliques duplicados e delegará o processo à thread de background. Após alguns segundos (tempo variável de acordo com o hardware de GPU/CPU local para processamento dos tokens da IA), o sistema exibirá automaticamente:
 - O código SQL puro e formatado estruturado pelo modelo local na caixa de saída de código.

- Os dados correspondentes do banco populados de forma tabular na tabela central de resultados da aplicação.
- Mensagens informativas de auditoria no console de logs inferior confirmando o sucesso do pipeline.

8.1 Avisos e Limitações de Uso

Embora a aplicação tenha apresentado resultados satisfatórios durante os testes realizados, algumas limitações inerentes ao uso de Modelos de Linguagem de Grande Escala (LLMs) devem ser consideradas.

- O sistema depende exclusivamente das tabelas e colunas existentes no banco de dados conectado. Perguntas que façam referência a entidades inexistentes, tabelas ausentes ou informações não armazenadas podem resultar em consultas SQL inválidas ou respostas incorretas.
- Consultas excessivamente complexas, envolvendo múltiplos níveis de agregação, subconsultas aninhadas, expressões específicas de um SGBD ou lógica analítica avançada, podem levar o modelo a gerar instruções SQL incompatíveis com o banco utilizado e normalmente consomem muito tempo, obviamente variando conforme a capacidade de processamento local.
- Em cenários mais complexos, o tempo de geração da consulta pode aumentar significativamente, uma vez que o modelo precisa interpretar a pergunta e construir a instrução SQL correspondente. Em alguns casos, isso pode causar atrasos perceptíveis na interface.
- O sistema não realiza validação semântica completa da consulta gerada antes da execução. Dessa forma, erros de sintaxe ou incompatibilidades com o SGBD podem ocorrer e serão reportados ao usuário na caixa de mensagens.
- Recomenda-se que as perguntas sejam formuladas utilizando linguagem clara e objetiva. O uso de gírias, abreviações incomuns, regionalismos ou vocabulário excessivamente específico de determinados contextos pode reduzir a capacidade de interpretação do modelo.
- Sinônimos amplamente conhecidos normalmente são compreendidos pelo modelo, porém termos ambíguos ou pouco utilizados podem resultar em interpretações incorretas e, conseqüentemente, em consultas SQL inadequadas.
- O desempenho e a qualidade das consultas geradas dependem diretamente da capacidade do modelo utilizado e da qualidade das informações presentes no esquema do banco de dados.

9 Validação Experimental

Com o objetivo de avaliar a capacidade de geração automática de consultas SQL, foi utilizada uma base de dados acadêmica baseada no esquema *University*, amplamente empregada em disciplinas introdutórias de Banco de Dados.

Foram realizados aproximadamente 37 testes envolvendo diferentes níveis de complexidade e cobrindo os principais recursos da linguagem SQL.

Tabela 3: Categorias de funcionalidades avaliadas

Categoria	Ocorrências
Consultas simples (SELECT)	8
Filtros (WHERE)	7
Ordenação (ORDER BY)	4
Junções (JOIN)	10
Agrupamentos (GROUP BY)	8
Funções de agregação	8
DISTINCT	2
Relacionamentos entre múltiplas tabelas	10
Consultas com chaves estrangeiras compostas	5

Os testes incluíram consultas envolvendo junções simples e múltiplas, agrupamentos, funções de agregação, ordenações, filtros condicionais e relacionamentos complexos entre tabelas.

Na maior parte dos cenários avaliados, o modelo foi capaz de gerar consultas semanticamente corretas e compatíveis com a estrutura da base de dados, retornando os resultados esperados após execução no banco.

Além da validação sintática das consultas geradas, foi realizada conferência manual dos dados retornados para verificar se a interpretação da linguagem natural permanecia alinhada com a intenção original da pergunta.

Entretanto, os experimentos também evidenciaram limitações do modelo em consultas analíticas de elevada complexidade. Para avaliar esse comportamento, foi realizado um teste baseado na consulta Q1 do benchmark TPC-H, amplamente utilizado para avaliação de sistemas de banco de dados.

A solicitação realizada ao sistema foi:

Considere uma tabela de itens de linha de pedidos contendo informações sobre cada item enviado, incluindo a data de envio, a quantidade, o valor bruto por item, o percentual de desconto aplicado, a taxa de imposto e dois campos categóricos que indicam, respectivamente, o status de devolução e o status do processamento do item.

Você deve produzir uma consulta analítica que agregue esses dados por essas duas categorias de status, calculando diferentes métricas estatísticas e financeiras.

A análise deve considerar apenas os registros cujo envio tenha ocorrido até 90 dias antes da data de referência 01/12/1998.

Para cada combinação dos dois status, calcule:

- total de unidades;
- valor total bruto;
- valor total com desconto aplicado;

- valor total final incluindo imposto;
- média da quantidade;
- média do valor bruto;
- média do desconto;
- quantidade total de registros.

Por fim, agrupe e ordene o resultado pelos dois campos de status.

A consulta gerada pelo modelo foi:

```

1 SELECT
2     l_returnflag,
3     l_linestatus,
4     SUM(l_quantity) AS sum_qty,
5     SUM(l_extendedprice) AS sum_base_price,
6     SUM(l_extendedprice * (1 - l_discount))
7     AS sum_disc_price,
8     SUM(l_extendedprice * (1 - l_discount)
9     * (1 + l_tax))
10    AS sum_charge,
11     AVG(l_quantity) AS avg_qty,
12     AVG(l_extendedprice) AS avg_price,
13     AVG(l_discount) AS avg_disc,
14     COUNT(*) AS count_order
15 FROM lineitem
16 WHERE l_shipdate <=
17     DATE '1998-12-01'
18     - INTERVAL '90' DAY
19 GROUP BY
20     l_returnflag,
21     l_linestatus
22 ORDER BY
23     l_returnflag,
24     l_linestatus;

```

Durante a execução, o banco de dados MySQL retornou erro de sintaxe:

```

1 (pymysql.err.ProgrammingError)
2 (1064, "You have an error in your SQL syntax...")

```

A principal causa do problema foi a utilização de uma sintaxe de manipulação de datas compatível com outros dialetos SQL, mas incompatível com o MySQL utilizado durante os testes. Embora a estrutura geral da consulta, os agrupamentos, agregações e ordenações tenham sido gerados corretamente, o modelo não conseguiu adaptar adequadamente a expressão temporal ao dialeto específico do banco conectado.

Esse resultado evidencia uma limitação importante da abordagem adotada: apesar de o modelo demonstrar elevada capacidade de compreensão semântica da pergunta e da estrutura do banco, ainda podem ocorrer erros sintáticos em consultas avançadas, especialmente quando envolvem diferenças entre dialetos SQL ou construções analíticas complexas.

Assim, os experimentos indicam que o sistema é adequado para consultas acadêmicas e corporativas de baixa e média complexidade, mas consultas analíticas sofisticadas ainda podem exigir validação humana antes da execução em ambientes de produção.

10 Conclusão

O desenvolvimento do sistema *NL to SQL Converter* demonstrou a viabilidade da integração entre interfaces gráficas desenvolvidas em PySide6, mecanismos de acesso a bancos de dados baseados em SQLAlchemy e modelos locais de linguagem executados por meio do Ollama.

A arquitetura modular adotada permitiu separar claramente as responsabilidades entre interface gráfica, conexão com bancos de dados, extração automática de esquema, geração de consultas SQL e execução das instruções geradas. Além disso, o uso de processamento assíncrono com `QThread` garantiu que a interface permanecesse responsiva durante operações potencialmente demoradas, como a inferência do modelo de linguagem e a execução de consultas no banco.

Os experimentos realizados indicaram que o modelo `qwen3:14b` apresentou bom desempenho na geração automática de consultas SQL para problemas de baixa e média complexidade, sendo capaz de interpretar corretamente estruturas relacionais, relacionamentos entre tabelas, filtros, agrupamentos, funções de agregação e diferentes tipos de junções. A elevada taxa de acerto observada nos testes evidencia o potencial da abordagem Text-to-SQL para simplificar o acesso a bancos de dados por usuários sem conhecimento aprofundado da linguagem SQL.

Entretanto, os resultados também evidenciaram limitações inerentes ao uso de modelos de linguagem. Em cenários analíticos mais complexos, especialmente aqueles envolvendo funções avançadas, operações temporais ou diferenças entre dialetos SQL, foram observadas falhas sintáticas e incompatibilidades que exigem validação humana antes da execução em ambientes críticos. Dessa forma, embora o sistema seja capaz de automatizar grande parte do processo de construção de consultas, ele não elimina completamente a necessidade de supervisão do usuário.

De maneira geral, os objetivos propostos para o projeto foram alcançados. O sistema desenvolvido é capaz de converter perguntas em linguagem natural para SQL, executar as consultas geradas e apresentar os resultados ao usuário de forma integrada. Além de servir como uma ferramenta prática para exploração de bases de dados, o projeto também demonstra a aplicação de modelos de linguagem em problemas reais de bancos de dados e interfaces inteligentes.

O código-fonte completo, documentação e histórico de desenvolvimento encontram-se disponíveis no repositório oficial do projeto:

https://github.com/JoaoVictor-ec/nl_to_sql_llm